

Name: M Pradeep

Hall ticket No: 2303A51821

Batch: 26

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING																		
Program Name: B. Tech		Assignment Type: Lab	Academic Year: 2025-2026																	
Course Coordinator Name		Dr. Rishabh Mittal																		
Instructor(s) Name		<table border="1"> <tr><td>Mr. S Naresh Kumar</td></tr> <tr><td>Ms. B. Swathi</td></tr> <tr><td>Dr. Sasanko Shekhar Gantayat</td></tr> <tr><td>Mr. Md Sallauddin</td></tr> <tr><td>Dr. Mathivanan</td></tr> <tr><td>Mr. Y Srikanth</td></tr> <tr><td>Ms. N Shilpa</td></tr> <tr><td>Dr. Rishabh Mittal (Coordinator)</td></tr> <tr><td>Dr. R. Prashant Kumar</td></tr> <tr><td>Mr. Ankushavali MD</td></tr> <tr><td>Mr. B Viswanath</td></tr> <tr><td>Ms. Sujitha Reddy</td></tr> <tr><td>Ms. A. Anitha</td></tr> <tr><td>Ms. M.Madhuri</td></tr> <tr><td>Ms. Katherashala Swetha</td></tr> <tr><td>Ms. Velpula sumalatha</td></tr> <tr><td>Mr. Bingi Raju</td></tr> </table>		Mr. S Naresh Kumar	Ms. B. Swathi	Dr. Sasanko Shekhar Gantayat	Mr. Md Sallauddin	Dr. Mathivanan	Mr. Y Srikanth	Ms. N Shilpa	Dr. Rishabh Mittal (Coordinator)	Dr. R. Prashant Kumar	Mr. Ankushavali MD	Mr. B Viswanath	Ms. Sujitha Reddy	Ms. A. Anitha	Ms. M.Madhuri	Ms. Katherashala Swetha	Ms. Velpula sumalatha	Mr. Bingi Raju
Mr. S Naresh Kumar																				
Ms. B. Swathi																				
Dr. Sasanko Shekhar Gantayat																				
Mr. Md Sallauddin																				
Dr. Mathivanan																				
Mr. Y Srikanth																				
Ms. N Shilpa																				
Dr. Rishabh Mittal (Coordinator)																				
Dr. R. Prashant Kumar																				
Mr. Ankushavali MD																				
Mr. B Viswanath																				
Ms. Sujitha Reddy																				
Ms. A. Anitha																				
Ms. M.Madhuri																				
Ms. Katherashala Swetha																				
Ms. Velpula sumalatha																				
Mr. Bingi Raju																				
CourseCode	23CS002PC304	Course Title	AI Assisted Coding																	
Year/Sem	III/II	Regulation	R23																	
Date and Day of Assignment	Week9 – Wednesday	Time(s)	23CSBTB01 To 23CSBTB52																	
Duration	2 Hours	Applicable to Batches	All batches																	
Assignment Number: 17.3(Present assignment number)/24(Total number of assignments)																				
Q.No.	Question	Expected Time to complete																		
1	Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing Scripts	Week9 – Wednesday																		

Lab Objectives:

- Learn how to clean raw datasets using AI-assisted Python scripting.
- Apply preprocessing techniques such as handling missing values, encoding categorical data, and normalization.
- Automate repetitive data-cleaning tasks with AI-generated code.
- Understand how preprocessing impacts model performance.

Task 1 Employee Dataset Cleaning

Task:
Use AI assistance to clean an employee information dataset.

- Instructions:**
- Handle missing values in salary, designation, and experience
 - Remove duplicate employee IDs
 - Standardize text values (job titles in lowercase)
 - Encode categorical features (designation, employment_type)

Expected Output:
A clean dataset suitable for HR analytics and prediction tasks.

```
index.html | style.css | script.js | layout.html | layoutstyle.css | data.html | data.js | data.css | Skill_tracker.py | classnode.py X
C:\Users\pradeep> python classnode.py
1 import pandas as pd
2 from sklearn.preprocessing import LabelEncoder
3
4 # Sample employee dataset (replace with your actual data loading)
5 data = {
6     'employee_id': [1, 2, 3, 4, 5, 1, 6],
7     'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Alice', 'Frank'],
8     'salary': [50000, None, 60000, 55000, None, 50000, 65000],
9     'designation': ['Manager', 'Developer', None, 'manager', 'Analyst', 'Manager', 'developer'],
10    'experience': [5, 3, None, 4, 2, 5, 6],
11    'employment_type': ['Full-time', 'Part-time', 'Full-time', 'Contract', 'Full-time', 'Full-time', 'Part-time']
12 }
13
14 df = pd.DataFrame(data)
15
16 # Step 1: Remove duplicate employee IDs (keep the first occurrence)
17 df = df.drop_duplicates(subset='employee_id', keep='first')
18
19 # Step 2: Handle missing values
20 # For salary and experience, fill with median
21 df['salary'] = df['salary'].fillna(df['salary'].median())
22 df['experience'] = df['experience'].fillna(df['experience'].median())
23
24 # For designation: fill with mode (most frequent)
25 df['designation'] = df['designation'].fillna(df['designation'].mode()[0])
26
27 # Step 3: Standardize text values (job titles in lowercase)
28 df['designation'] = df['designation'].str.lower()
29
30 # Step 4: Encode categorical features
31 le_designation = LabelEncoder()
32 df['designation_encoded'] = le_designation.fit_transform(df['designation'])
33
34 le_employment = LabelEncoder()
35 df['employment_type_encoded'] = le_employment.fit_transform(df['employment_type'])
36
37 # Display the clean dataset
38 print(df)
39
40 # Optional: Save to CSV
41 df.to_csv('clean_employee_dataset.csv', index=False)
```



```

C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:/Users/prade/OneDrive/Pradeep/python language/classnode.py"
Preprocessed Data:
student_id exam_date month semester total_marks total_marks_normalized
0 1 2023-02-21 2 1 54.799829 0.181188
1 2 2023-04-03 4 1 61.291131 0.306131
2 3 2023-01-15 1 1 58.841715 0.239148
3 4 2023-03-13 3 1 77.816433 0.578383
4 5 2023-03-02 3 1 83.731842 0.698443
5 7 2023-03-24 3 1 88.414985 0.782169
6 8 2023-03-28 3 1 86.324967 0.744884
7 9 2023-03-16 3 1 71.892512 0.486776
8 10 2023-03-16 3 1 65.647839 0.375132
9 11 2023-03-29 3 1 52.377701 0.137885

Original shape: (100,), Processed shape: (97, 9)
6 8 2023-03-28 3 1 86.324967 0.744884
7 9 2023-03-16 3 1 71.892512 0.486776
8 10 2023-03-16 3 1 65.647839 0.375132
9 11 2023-03-29 3 1 52.377701 0.137885

Original shape: (100,), Processed shape: (97, 9)
8 2023-03-28 3 1 86.324967 0.744884
9 2023-03-16 3 1 71.892512 0.486776
10 2023-03-16 3 1 65.647839 0.375132
11 2023-03-29 3 1 52.377701 0.137885

Original shape: (100,), Processed shape: (97, 9)
Original shape: (100,), Processed shape: (97, 9)

Normalized marks range: [0.0000, 1.0000]

✓ Processed data saved to processed_student_data.csv

```

Task 3 – Student Health Data Preparation

Task:

Clean and preprocess student health records using AI scripts.

Instructions:

- Handle missing values in height, weight, BMI
- Encode categorical features (medical_condition, fitness_status)
- Scale numerical features using standard scaling
- Split dataset into training and testing sets

Expected Output:

A preprocessed dataset ready for predictive health analysis.

```

C:\Users\prade>OneDrive>Pradeep>python language>classnode.py
1 # Generate Sample Health Data
2 np.random.seed(42)
3 df = pd.DataFrame()
4
5 # Student ID (range 1, 101)
6 df['student_id'] = np.arange(1, 101)
7 # Age (random normal, 18, 30)
8 df['age'] = np.random.normal(24, 5, 100)
9 # Height (random normal, 170, 185)
10 df['height'] = np.random.normal(177.5, 10, 100)
11 # Weight (random normal, 70, 85)
12 df['weight'] = np.random.normal(77.5, 10, 100)
13 # BMI (random normal, 24, 30)
14 df['bmi'] = np.random.normal(27, 3, 100)
15 # Medical Condition (categorical: Healthy, Asthma, Diabetes, None)
16 df['medical_condition'] = np.random.choice(['Healthy', 'Asthma', 'Diabetes', 'None'], 100)
17 # Fitness Status (categorical: Excellent, Good, Fair, Poor, None)
18 df['fitness_status'] = np.random.choice(['Excellent', 'Good', 'Fair', 'Poor', 'None'], 100)
19 # Exercise Hours (random uniform, 0, 12)
20 df['exercise_hours'] = np.random.uniform(0, 12, 100)
21
22 # Add missing values
23 missing_idx = np.random.choice(100, 10, replace=False)
24 df.loc[missing_idx, 'height'] = np.nan
25 df.loc[missing_idx, 'weight'] = np.nan
26 df.loc[missing_idx, 'bmi'] = np.nan
27
28 print("Before preprocessing:")
29 print(df[missing_idx].sum())
30
31 # 1. Handle missing values
32 df['height'].fillna(df['height'].mean(), inplace=True)
33 df['weight'].fillna(df['weight'].mean(), inplace=True)
34 df['bmi'].fillna(df['bmi'].mean(), inplace=True)
35 df['medical_condition'].fillna('None', inplace=True)
36 df['fitness_status'].fillna('Fair', inplace=True)
37
38 # 2. Encode categorical features
39 le_medical = LabelEncoder()
40 df['medical_encoded'] = le_medical.fit_transform(df['medical_condition'])
41 le_fitness = LabelEncoder()
42 df['fitness_encoded'] = le_fitness.fit_transform(df['fitness_status'])
43
44 # 3. Scale numerical features
45 numerical_cols = ['age', 'height', 'weight', 'bmi', 'exercise_hours']
46 scaler = StandardScaler()
47 df[numerical_cols] = scaler.fit_transform(df[numerical_cols])
48
49 # 4. Split dataset
50 X = df[numerical_cols + ['medical_encoded', 'fitness_encoded']]
51 y = df['bmi']
52 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
53
54 print("After preprocessing:")
55 print(df[missing_idx].sum())
56 print(f"Training set: {len(X_train)}, Testing set: {len(X_test)}")
57 print(f"Test set: {len(X_test)}")
58
59 # Save
60 df.to_csv('c:/Users/prade/OneDrive/Pradeep/python language/preprocessed_health_data.csv', index=False)
61 X_train.to_csv('c:/Users/prade/OneDrive/Pradeep/python language/health_train.csv', index=False)
62 X_test.to_csv('c:/Users/prade/OneDrive/Pradeep/python language/health_test.csv', index=False)
63
64 print("\n Data saved: processed_health_data.csv, health_train.csv, health_test.csv")
65
C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:/Users/prade/OneDrive/Pradeep/python language/classnode.py"
Before preprocessing:
Missing values:
student_id      0
age             0
height          3
weight          3
bmi             4
medical_condition 21
fitness_status  25
exercise_hours  0
dtype: int64

```

See the documentation for a more detailed explanation: https://pandas.pydata.org/pandas-docs/stable/user_guide/copy_on_write.html

```
df['fitness_status'].fillna('Fair', inplace=True)
After preprocessing:
Missing values: 56
Encoded features: 2
Scaled features: 5

Training set: 80, Testing set: 20

   age  height  weight    bmi  exercise_hours  medical_encoded  fitness_encoded
55  1.465198  0.306641  0.023128  0.300740      -0.473295         1         2
88  0.414877 -1.596923 -0.885985 -0.042759      0.560665         3         1
26 -0.110284  0.412532 -0.387245 -0.502634      0.135869         3         4
42  1.465198 -0.817847 -0.364701  2.263329     -1.788820         1         0
69  1.465198  0.004662  1.032667  1.151545     -1.403043         2         2

✓ Data saved: processed_health_data.csv, health_train.csv, health_test.csv
After preprocessing:
Missing values: 56
Encoded features: 2
Scaled features: 5

Training set: 80, Testing set: 20

   age  height  weight    bmi  exercise_hours  medical_encoded  fitness_encoded
55  1.465198  0.306641  0.023128  0.300740      -0.473295         1         2
88  0.414877 -1.596923 -0.885985 -0.042759      0.560665         3         1
26 -0.110284  0.412532 -0.387245 -0.502634      0.135869         3         4
42  1.465198 -0.817847 -0.364701  2.263329     -1.788820         1         0
69  1.465198  0.004662  1.032667  1.151545     -1.403043         2         2

✓ Data saved: processed_health_data.csv, health_train.csv, health_test.csv
✓ Data saved: processed_health_data.csv, health_train.csv, health_test.csv
```

Task 4: Real-Time Application: Ride-Sharing Data Cleaning Scenario:

A ride-sharing company has trip data with errors.

Requirements:

- Standardize pickup and drop locations
- Fill missing fare amounts with median values
- Remove duplicate ride entries
- Normalize driver ratings
- Generate a summary of data improvements

Expected Output:

A clean dataset ready for operational analysis.

```
C:\Users\prade> CodeRunner > Run (Python language) > C:\code\code.py
6 # Generate Ride-Sharing Data
7 np.random.seed(42)
8 data = {
9     'ride_id': range(1, 101),
10    'pickup_location': np.random.choice(['Downtown', 'Airport', 'Downtown', 'Airport', None], 100),
11    'drop_location': np.random.choice(['Midtown', 'Station', 'Station', 'Midtown', None], 100),
12    'fare_amount': np.random.normal(20, 10, 100),
13    'driver_rating': np.random.uniform(1, 5, 100),
14 }
15
16 df = pd.DataFrame(data)
17 # Add duplicate rows and missing values
18 df.loc[random_rows] = np.random.choice(100, 5, replace=False), 'fare_amount' = np.nan
19 df = pd.concat([df, df.iloc[15]], ignore_index=True)
20
21 print("RIDE-SHARING DATA CLEANING PIPELINE")
22 print("="*40)
23 print(f"Original shape: {df.shape}")
24 print(f"Missing values: {df.isnull().sum()}")
25
26 # 1. Standardize Locations
27 df['pickup_location'] = df['pickup_location'].str.strip().str.lower().str.title()
28 df['drop_location'] = df['drop_location'].str.strip().str.lower().str.title()
29 df['pickup_location'].fillna('Unknown', inplace=True)
30 df['drop_location'].fillna('Unknown', inplace=True)
31 print(f"✓ Standardized pickup and drop locations")
32
33 # 2. Fill Missing Fares with Median
34 median_fare = df['fare_amount'].median()
35 df['fare_amount'].fillna(median_fare, inplace=True)
36 print(f"✓ Filled missing fares with median: {median_fare:.1f}")
37
38 # 3. Remove Duplicates
39 initial_rows = len(df)
40 df = df.drop_duplicates(subset=['ride_id', 'pickup_location', 'drop_location'], keep='first')
41 df = df.reset_index(drop=True)
42 duplicates_removed = initial_rows - len(df)
43 print(f"✓ Removed duplicate entries: {duplicates_removed} rows")
44
45 # 4. Normalize Driver Ratings (0-1 scale)
46 scaler = MinMaxScaler()
47 df['driver_rating_normalized'] = scaler.fit_transform(df[['driver_rating']])
48 print(f"✓ Normalized driver ratings (0-1 scale)")
49
50 # 5. Generate Summary
51 print("\n" + "="*40)
52 print("DATA QUALITY IMPROVEMENT SUMMARY")
53 print("="*40)
54 print(f"Final dataset shape: {df.shape}")
55 print(f"Duplicate entries removed: {duplicates_removed}")
56 print(f"Missing fares filled: {(df['fare_amount'].isnull().sum() == 0)}")
57 print(f"Location standardization: Complete")
58 print(f"Driver ratings normalized: Complete")
59 print(f"✓ Fare Statistics:")
60 print(f"  Min: {df['fare_amount'].min():.1f}")
61 print(f"  Max: {df['fare_amount'].max():.1f}")
62 print(f"  Mean: {df['fare_amount'].mean():.1f}")
63 print(f"  Median: {df['fare_amount'].median():.1f}")
```

```
index.html style.css script.js layout.html layoutstyle.css cta.html cta.js cta.css Skill_tracker.py classnode.py X
C:\Users\prade > OneDrive > Pradeep > python language > classnode.py > ...

64
65 # Save cleaned data
66 df.to_csv('c:/Users/prade/OneDrive/Pradeep/python language/cleaned_rides.csv', index=False)
67 print("\n/ Clean dataset saved to cleaned_rides.csv")
68 print("\nCleaned Data Sample:\n(df.head())")
69

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER
C:\Users\prade>C:\Users\prade\AppData\Local\Microsoft\WindowsApps\python3.12.exe "c:/Users/prade/OneDrive/Pradeep/python language/classnode.py"
RIDE-SHARING DATA CLEANING PIPELINE
=====

Original shape: (185, 5)
Missing values:
ride_id      0
pickup_location 22
drop_location 17
fare_amount  5
driver_rating 0
dtype: int64

=====
DATA QUALITY IMPROVEMENT SUMMARY
=====
df[('fare_amount')]fillna(median_fare, inplace=True)
/ Filled missing fares with median: $25.44
/ Removed duplicate entries: 5 rows
/ Normalized driver ratings (0-1 scale)

=====
DATA QUALITY IMPROVEMENT SUMMARY
=====
Final dataset shape: (180, 6)
Duplicate entries removed: 5
Missing fares filled: false
Location standardization: Complete
Driver ratings normalized: Complete

Fare Statistics:
Min: $1.62
Max: $55.41
Mean: $25.38
Median: $25.28
df[('fare_amount')]fillna(median_fare, inplace=True)
/ Filled missing fares with median: $25.44
/ Removed duplicate entries: 5 rows
/ Normalized driver ratings (0-1 scale)

=====
DATA QUALITY IMPROVEMENT SUMMARY
=====
Final dataset shape: (180, 6)
Duplicate entries removed: 5
Missing fares filled: false
Location standardization: Complete
Driver ratings normalized: Complete

Fare Statistics:
Min: $1.62
Max: $55.41
Mean: $25.38
Median: $25.28
df[('fare_amount')]fillna(median_fare, inplace=True)
/ Filled missing fares with median: $25.44
/ Removed duplicate entries: 5 rows
/ Normalized driver ratings (0-1 scale)

=====
DATA QUALITY IMPROVEMENT SUMMARY
=====
Final dataset shape: (180, 6)
Duplicate entries removed: 5
Missing fares filled: false
Location standardization: Complete
Driver ratings normalized: Complete

Fare Statistics:
Min: $1.62
Max: $55.41
Mean: $25.38
Median: $25.28
/ Clean dataset saved to cleaned_rides.csv

Fare Statistics:
Min: $1.62
Max: $55.41
Mean: $25.38
Median: $25.28
/ Clean dataset saved to cleaned_rides.csv

Fare Statistics:
Min: $1.62
Max: $55.41
Mean: $25.38
Median: $25.28
/ Clean dataset saved to cleaned_rides.csv

Cleaned Data Sample:

Cleaned Data Sample:
ride_id pickup_location drop_location fare_amount driver_rating driver_rating_normalized
ride_id pickup_location drop_location fare_amount driver_rating driver_rating_normalized
0 1 Airport Midtown 24.545971 2.312611 0.323680
0 1 Airport Midtown 24.545973 2.312611 0.323680
1 2 NaN Midtown 27.169367 1.628166 0.144575
2 3 Downtown Midtown 30.124237 4.927364 1.000000
1 2 NaN Midtown 27.169367 1.628166 0.144575
2 3 Downtown Midtown 30.124237 4.927364 1.000000
3 4 NaN Station 30.434887 4.355734 0.852145
4 5 NaN Midtown 25.280995 4.441618 0.874359
```

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.